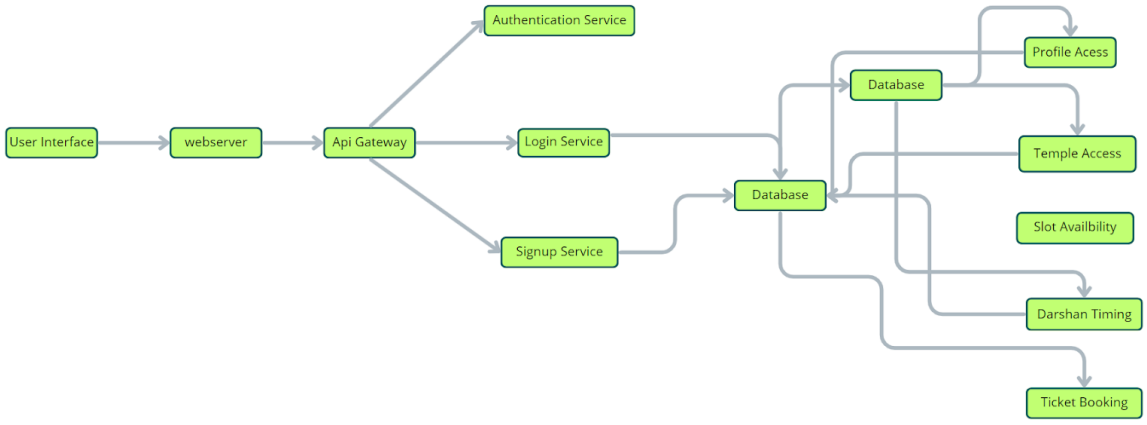


In this technical architecture, the DarshanEase consists of several components:

User Interface: Represents the web interface through which devotees interact with the system to explore darshan timings, select preferred slots, and make bookings.

Web Server: Hosts the user interface and serves the web pages to the users.

API Gateway: Acts as a single entry point for client requests and routes them to the respective services.

Authentication Service: Handles user authentication and authorization for secure access to the system.

Database: Stores persistent data related to darshan slots, bookings, payments, and other entities crucial to the temple darshan ticket booking system.

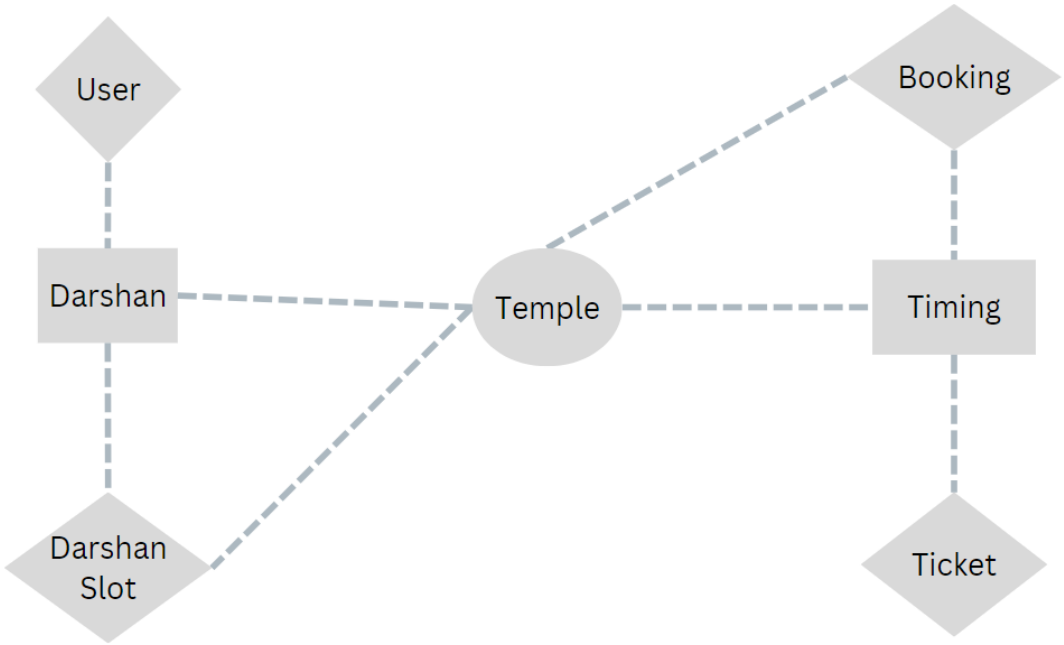
Darshan Service: Manages temple-related information, including darshan timings, slot availability, and temple access details.

Booking Service: Handles the process of creating and managing darshan bookings, allowing devotees to secure slots for their temple visits.

Temple Service: Manages information about temples, including their locations, darshan amenities, and access details.

Seat Service: Manages seat availability, selection, and reservation for devotees attending temple darshans.

Ticket Service: Generates and manages electronic tickets for devotees, providing a digital record of their darshan bookings.



User:

UserID (PK)

Name

Email

Phone

Address

Temple:

TempleID (PK)

TempleName

Location

DarshanStartTime

DarshanEndTime

DarshanSlot:

SlotID (PK)

TempleID (FK)

Date

StartTime

EndTime

AvailableSeats

Price

Booking:

BookingID (PK)

UserID (FK)

SlotID (FK)

BookingDate

TotalAmount

The relationships between the entities are as follows:

User-Booking Relationship: One-to-Many relationship between User and Booking (One user can make multiple bookings)

Foreign Key: UserID in Booking

Temple-DarshanSlot Relationship: One-to-Many relationship between Temple and DarshanSlot (One temple can have multiple darshan slots)

Foreign Key: TempleID in DarshanSlot

Booking-DarshanSlot Relationship: One-to-Many relationship between DarshanSlot and Booking (One darshan slot can have multiple bookings)

Foreign Keys: SlotID in Booking, TempleID in DarshanSlot

The foreign key relationships are established between the entities using the primary keys from the respective tables.

Please note that this is just an example, and you can modify or expand it based on your specific requirements for the DarshanEase.

FEATURES

Devotee Registration and Authentication: Enable devotees to register accounts, log in securely, and authenticate their identity to access the temple darshan ticket booking system.

Darshan Listings: Display a comprehensive list of available darshan slots with details such as temple name, location, darshan date, and timings.

Temple Selection: Provide a selection of temples offering darshan slots, including detailed information about each temple's location and unique darshan offerings.

Slot Selection: Allow devotees to choose their preferred darshan slots from the available options, showcasing the layout of the temple and available slots.

Ticket Booking: Empower devotees to book tickets for a specific darshan slot, specifying the number of devotees accompanying them.

Donation Integration: Integrate with a secure payment gateway to facilitate online donations and contributions along with the darshan ticket booking.

Booking Confirmation: Provide devotees with a confirmation page or email containing the details of the darshan booking, including darshan slot information and booking ID.

Booking History: Allow devotees to view their past and upcoming darshan bookings, offering options to cancel or modify bookings if allowed by the temple.

Darshan Slot Availability Tracking: Keep track of the availability of darshan slots for each temple and update in real-time as slots are booked or canceled.

•**Organizer Dashboard:** Provide temple Registration with an interface to manage temple, darshan slots, devotees, availability, and booking-related activities.

Darshan Slot Management: Organizer is responsible ticket issueing and slot booking.

Admin Dashboard: Provide temple administrators with an interface to manage users, organizers,temples, darshan slots, devotees, availability, and booking-related activities.

Reporting and Analytics: Generate reports and analytics on darshan bookings, popular temples, devotee demographics, and other relevant metrics to gain insights into the temple's activities.

Integration with External APIs: Integrate with third-party APIs for services like temple information, darshan slot updates, or real-time availability for seamless data synchronization.These are just a few key features, and you can customize and expand them based on your specific requirements and the scale of the ticket booking system you are building.

ROLES AND RESPONSIBILITIES

Description

User:-

Registration: Devotees are responsible for creating an account on the Temple Darshan Ticket Booking App by providing necessary details like name, email, and password.

Profile Management: Devotees have access to manipulate their profiles, allowing them to update information such as email, name, and password.

Darshan Booking: Devotees can view available darshan slots, book specific slots, and acquire electronic tickets for their temple visits.

Feedback: Provide feedback and ratings for the darshan experience.

Logout: Finally, they can logout from the DarshanEase.

Organizer:-

Profile Management: Temple organizers have access to manipulate their profiles, allowing them to update information such as email, name, and password.

Darshan Slot Management: Temple organizers are responsible for managing darshan slots, including scheduling, updating availability, and marking special slots.

Booking Management: Temple organizers can view and manage darshan bookings, assign slots, and make adjustments as necessary.

Notification Handling: Receive notifications related to darshan bookings and updates for efficient communication with devotees.

Logout: Finally, they can logout from the DarshaEase.

Admin:-

System Management: Admins have full control over all aspects of the temple darshan ticket booking system, overseeing functionalities, configurations, and security.

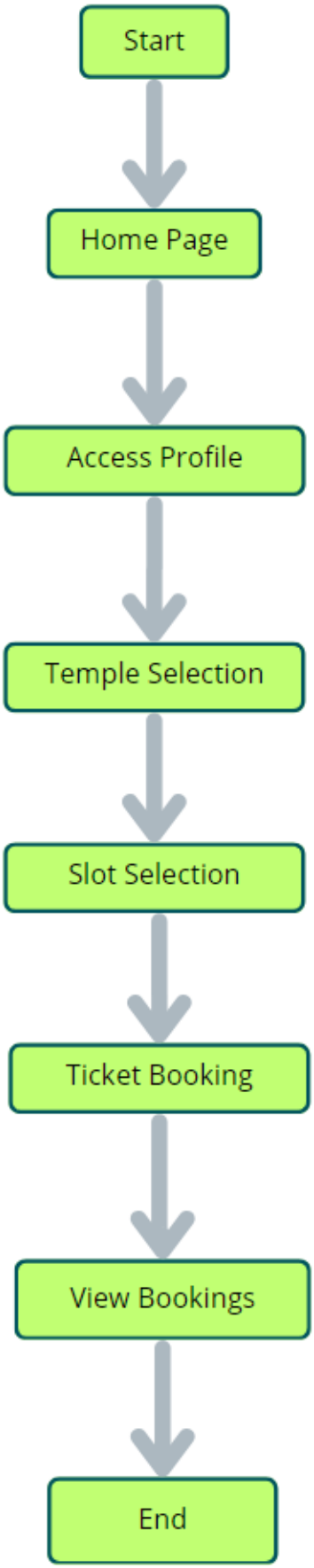
Devotee Management: Admins can manipulate devotee information, including creating, updating, and deleting accounts. They also have the authority to manage darshan ratings.

Temple Organizer Management: Admins can manipulate temple organizer information, including creating, updating, and deleting accounts.

Facility Management: Admins can schedule darshan slots, allocate resources efficiently, and handle maintenance requests for the temple.

Events Management: Admins have the authority to create and manage temple events, including scheduling and deleting events.

Logout: Finally, they can logout from the DarshaEase.


Start
Home Page
Access Profile
Temple Selection
Slot Selection
Ticket Booking
View Bookings
End

1. Start: The process begins as users enter the Temple Darshan Ticket Booking App.

2. Home Page: Users land on the home page, providing an overview of the temple darshan ticket booking services. From here, they can initiate various actions.

3. Access Profile: Users have the option to access their profiles, allowing them to view or update personal information, preferences, and past booking history.

4. Temple Selection:After accessing their profiles, users proceed to select a specific temple for darshan. The app presents a list of available temples, along with relevant details.

5. Slot Selection: Users navigate through the available darshan slots for the selected temple. They can view darshan timings, slot availability, and choose a preferred time slot for their visit.

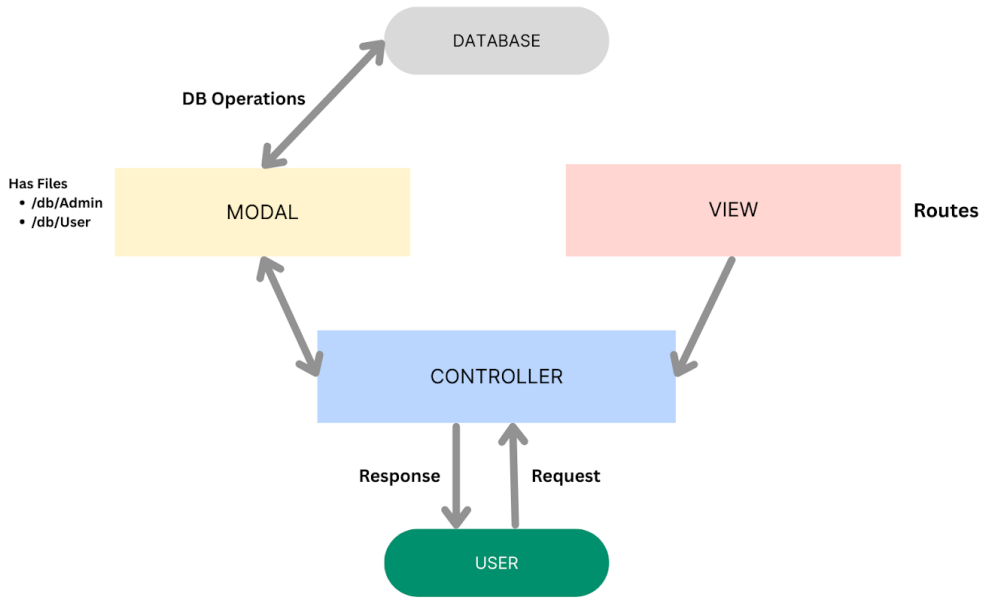
6. View Bookings: Users have the option to view their current and past darshan bookings. This section provides details about booked slots, confirmation status, and electronic tickets.

7. Ticket Booking: For new bookings, users can initiate the ticket booking process. This involves selecting a temple, choosing a darshan slot, confirming the booking, and receiving an electronic ticket.

8. End: The flow concludes as users have completed their desired actions within the

This flowchart outlines the user journey within the app, starting from the home page, navigating through profile and temple selections, managing bookings, and concluding with the booking process or other activities, ultimately providing a seamless experience for DarshanEase.

Feel free to modify the code as per your specific requirements or add additional nodes as needed.



The Darshan Ease backend application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.

Scalability: New features can be added easily by creating new routes, controllers, and models.

Reusability: Logic in controllers and models can be reused across multiple parts of the application.

Testing: Each layer can be tested independently, especially the controllers and models.

Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.